

Ejercicios Básicos de Angular

5 prácticas para aprender los fundamentos del framework

Este documento contiene cinco ejercicios progresivos pensados para quienes se inician en Angular. Cada ejercicio incluye el objetivo, los conceptos que se practican y el código de ejemplo necesario para resolverlo. Se recomienda tener instalado Node.js y el Angular CLI (**npm install -g @angular/cli**) antes de comenzar.

#	Ejercicio	Conceptos
1	Componente de saludo	Componentes, interpolación
2	Contador con botones	Eventos, property binding
3	Lista de tareas	*ngFor, ngModel, arrays
4	Mostrar/ocultar elementos	*ngIf, condicionales
5	Comunicación entre componentes	@Input, @Output

Ejercicio 1 · Componente de saludo

Objetivo: Crear un componente que muestre un mensaje de saludo utilizando interpolación de datos.

Conceptos: creación de componentes, interpolación `{{ }}`, propiedades de clase.

Comando para generar el componente:

```
ng generate component saludo
```

saludo.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-saludo',
  templateUrl: './saludo.component.html'
})
export class SaludoComponent {
  nombre: string = 'Mundo';

  cambiarNombre(): void {
    this.nombre = 'Angular';
  }
}
```

saludo.component.html

```
<h1>¡Hola, {{ nombre }}!</h1>
<button (click)="cambiarNombre()">Cambiar nombre</button>
```

Reto extra: añade un campo de texto para que el usuario escriba su propio nombre y se muestre en el saludo.

Ejercicio 2 · Contador con botones

Objetivo: Construir un contador que aumente o disminuya su valor al pulsar distintos botones.

Conceptos: manejo de eventos (click), property binding, métodos de clase.

contador.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-contador',
  templateUrl: './contador.component.html'
})
export class ContadorComponent {
  contador: number = 0;

  incrementar(): void {
    this.contador++;
  }

  decrementar(): void {
    this.contador--;
  }

  reiniciar(): void {
    this.contador = 0;
  }
}
```

contador.component.html

```
<h2>Contador: {{ contador }}</h2>
<button (click)="decrementar()">-</button>
<button (click)="reiniciar()">Reiniciar</button>
<button (click)="incrementar()">+</button>
```

Reto extra: deshabilita el botón de decrementar cuando el contador llegue a cero usando [disabled].

Ejercicio 3 · Lista de tareas

Objetivo: Crear una lista de tareas (to-do list) donde se puedan añadir nuevos elementos desde un formulario.

Conceptos: directiva estructural `*ngFor`, two-way binding con `[(ngModel)]`, arrays.

Recuerda importar `FormsModule` en el módulo para poder usar `ngModel`.

tareas.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-tareas',
  templateUrl: './tareas.component.html'
})
export class TareasComponent {
  nuevaTarea: string = '';
  tareas: string[] = ['Aprender Angular', 'Practicar TypeScript'];

  agregarTarea(): void {
    if (this.nuevaTarea.trim()) {
      this.tareas.push(this.nuevaTarea);
      this.nuevaTarea = '';
    }
  }

  eliminarTarea(indice: number): void {
    this.tareas.splice(indice, 1);
  }
}
```

tareas.component.html

```
<input [(ngModel)]="nuevaTarea" placeholder="Nueva tarea">
<button (click)="agregarTarea()">Agregar</button>

<ul>
  <li *ngFor="let tarea of tareas; let i = index">
    {{ tarea }}
    <button (click)="eliminarTarea(i)">Eliminar</button>
  </li>
</ul>
```

Reto extra: añade una casilla para marcar cada tarea como completada y tacha el texto cuando esté marcada.

Ejercicio 4 · Mostrar u ocultar elementos

Objetivo: Practicar la renderización condicional mostrando u ocultando bloques de contenido según una variable booleana.

Conceptos: directiva estructural *ngIf, operadores condicionales, interpolación con expresiones ternarias.

panel.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-panel',
  templateUrl: './panel.component.html'
})
export class PanelComponent {
  mostrarDetalles: boolean = false;

  alternar(): void {
    this.mostrarDetalles = !this.mostrarDetalles;
  }
}
```

panel.component.html

```
<button (click)="alternar()">
  {{ mostrarDetalles ? 'Ocultar' : 'Mostrar' }} detalles
</button>

<div *ngIf="mostrarDetalles">
  <p>Estos son los detalles adicionales que solo se ven
    cuando el usuario decide mostrarlos.</p>
</div>
```

Reto extra: usa *ngIf con else para mostrar un mensaje alternativo cuando los detalles estén ocultos.

Ejercicio 5 · Comunicación entre componentes

Objetivo: Crear un componente padre y uno hijo que se comuniquen entre sí mediante propiedades de entrada y eventos de salida.

Conceptos: decoradores `@Input()` y `@Output()`, `EventEmitter`.

tarjeta.component.ts (hijo)

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-tarjeta',
  templateUrl: './tarjeta.component.html'
})
export class TarjetaComponent {
  @Input() titulo: string = '';
  @Output() seleccionada = new EventEmitter<string>();

  seleccionar(): void {
    this.seleccionada.emit(this.titulo);
  }
}
```

tarjeta.component.html (hijo)

```
<div class="tarjeta" (click)="seleccionar()">
  <h3>{{ titulo }}</h3>
</div>
```

app.component.ts (padre)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  tarjetas: string[] = ['Angular', 'React', 'Vue'];
  seleccionActual: string = '';

  onSeleccion(titulo: string): void {
    this.seleccionActual = titulo;
  }
}
```

app.component.html (padre)

```
<app-tarjeta
  *ngFor="let t of tarjetas"
  [titulo]="t"
  (seleccionada)="onSeleccion($event)">
</app-tarjeta>

<p>Seleccionaste: {{ seleccionActual }}</p>
```

Reto extra: añade un segundo `@Output()` que emita un evento cuando el usuario pase el ratón por encima de la tarjeta.